Detecția stărilor de avarie în sisteme industriale folosind tehnici de învățare automată

LUCRARE DE LICENȚĂ

Student: **Andrea Marton**
Coordonator: **Lector univ. dr. Roland Bolboacă**

Târgu-Mureș, Iulie 2025

FACULTATEA DE INGINERIE ȘI TEHNOLOGIA INFORMAȚIEI
Programul de studii: INFORMATICĂ

Cuprins

1. Introducere în tematica lucrării
2. Aspecte teoretice
3. Metode utilizate
4. Implementare practică
5. Setul de date Tennessee Eastman Process
6. Rezultate experimentale
7. Concluzii și aplicație

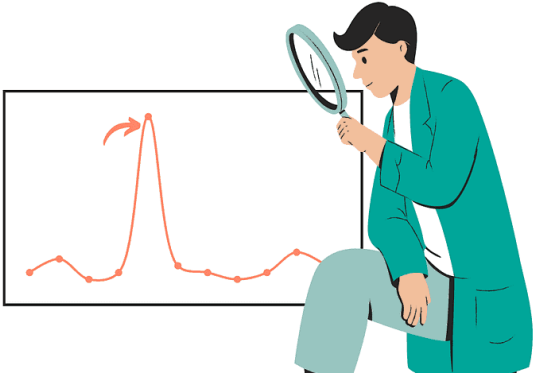
Introducere în tematica lucrării

Introducere în tematica lucrării

- Avaria = deviere de la starea normală $\rightarrow$ risc industrial
- Detectarea timpurie previne accidente și pierderi
- Soluție: învățare automată pentru detecția anomaliilor
- Două abordări analizate:
  - Rețea neuronală (supervizată)
  - Autoencoder + OCSVM (nesupervizată)

Aspecte teoretice

Ce este o anomalie?



Sursa imaginii: https://www.datameer.com/blog/4-must-know-techniques-for-effective-anomaly-detection-in-data-sets/

Aspecte teoretice - Anomalii

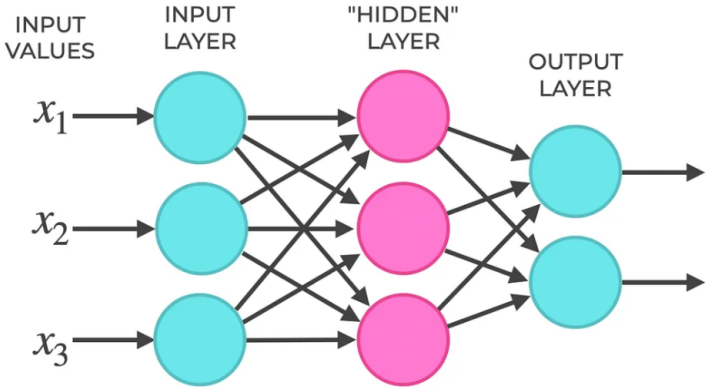
- Anomalie = abatere semnificativă față de comportamentul normal
- Detectarea anomaliilor este esențială în prevenirea avariilor
- Aplicații în domenii precum industria chimică, procese automatizate
- Lucrarea utilizează anomaliile pentru a identifica stările de avarie

Abordări în detectarea anomaliilor

- Supervizată - necesită date etichetate
- Nesupervizată - identifică tipare fără etichete
- Semi-supervizate - combină cele două
- În lucrare - comparare între supervizat și nesupevizat

Metode utilizate

Rețea neuronală



Sursa imaginii: `https://rajeevkdave.medium.com/multilayer-perceptron-de07c0ae6308`

Metoda supervizată - Rețea neuronală

- Rețea MLP (Mulilayer Perceptron)
- 2 straturi ascunse + strat de ieșire
- Funcții de activare ReLu și softmax
- Algoritm de antrenare: backpropagation + Adam
- Scop: Clasificarea între normal și avarie

Formule esențiale – Rețea MLP

- **Funcția de activare ReLU (straturi ascunse):**

$$f(x) = \max(0, x)$$

- **Funcția softmax (ieșire):**

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

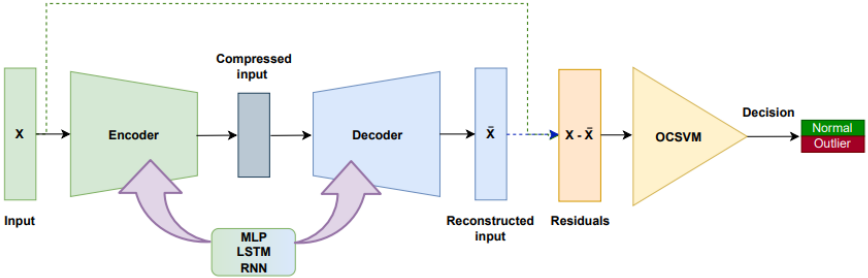
- **Funcția de pierdere (Cross-Entropy):**

$$L = -\sum_i y_i \cdot \log(\hat{y}_i)$$

- **Optimizare:** Gradient Descent cu Adam

Metoda nesupervizată - Autoencoder + OCSVM

- Ce este un autoencoder?



Sursa imaginii: `https://nislab.umfst.ro/files/INTERENG_ROLAND.pdf`

Metoda nesupervizată - Autoencoder + OCSVM

- Autoencoder $\rightarrow$ recunstruiește datele
- Se calculează eroarea de reconstrucție
- OCSVM (One Class Suport Vector Machine) $\rightarrow$ clasifică în "normal" sau "anormal"
- Nu are nevoie de etichete
- Ideal pentru date nestructurate sau necunoscute

Implementare practică

Descriere implementare

- Limbaj Python
- Biblioteci TensorFlow, Keras, scikit-learn
- Preprocesare normalizare, împărțiere în train/test
- Interfață grafică selecție model + epoci + afișare rezultate

Setul de date Tennessee Eastman Process

Setul de date

- Tennessee Eastman Process - simulare proces chimic
- Conține date normale și stări de avarie simulate
- Reprezintă un reper industrial foarte cunoscut în detecția anomaliilor
- Folosit pentru antrenarea și testarea modelelor

Rezultate experimentale

Rezultate

Caracteristică	Model 2 straturi	Model 3 straturi
Număr de epoci	50	50
Accuracy	0.8400	0.8629
TPR	0.2012	0.1646
Precision	0.2007	0.1692
F1 Score	0.2010	0.0962
FPR	0.0891	0.0932
TNR	0.9109	0.9016
FNR	0.7988	0.8376

- Acuratețe mai mare la 3 straturi
- TPR, Precizie și F1 Score mai bune la 2 straturi
- Rată mai mică de alarme false la 2 straturi

Rezultate

- Rețea neuronală - Precizie ridicată, dar necesită date etichetate
- Autoencoder + OCSVM - Mai flexibil, funcționează fără etichete
- Comparație - Modele complementare în funcție de tipul de date

Rezultate

Table 1: Compararea performanței dintre MLP și autoencoder la diverse valori ale numărului de epoci

Model	Epoci	Accuracy	Recall	Precision	F1 Score	FPR	TNR	FNR
MLP	20	0.9912	0.9081	1.0000	0.9518	0.0000	1.0000	0.0919
MLP	50	0.9926	0.9227	1.0000	0.9598	0.0000	1.0000	0.0773
MLP	100	0.9928	0.9251	1.0000	0.9611	0.0000	1.0000	0.0749
Autoencoder	20	0.8413	0.2084	0.2077	0.2081	0.0883	0.9117	0.7916
Autoencoder	50	0.8241	0.2088	0.2078	0.2083	0.0884	0.9116	0.7912
Autoencoder	100	0.8329	0.1665	0.1659	0.1662	0.0930	0.9070	0.8335

Rezultate

Table 2: Compararea performanței modelului MLP cu Autoencoder pe un set de date doar cu anomalii

Model	MLP	Autoencoder
Accuracy	0.9970	0.0990
Recall	0.9976	0.0990
Precision	0.9993	1.0000
F1 Score	0.9984	0.1801

Concluzii și aplicație

Aplicație demonstrativă - GUI

- Încărcare fișier cu date .csv
- Alegere metodă - supervizat/ nesupervizat
- Configurare epoci
- Vizualizare rezultate în format tabelar și grafic

GUI

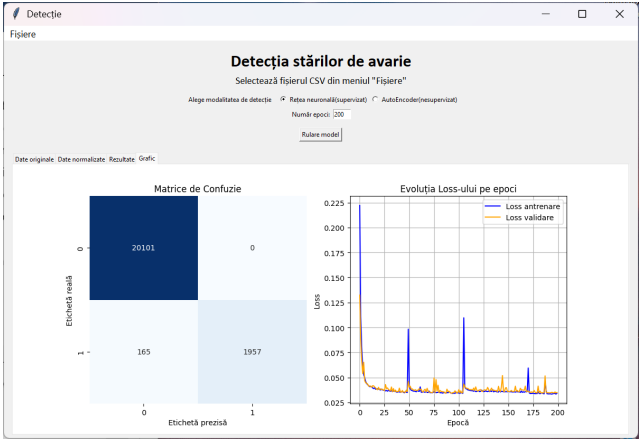


Figure 1: Interfața rezultatelor sub formă grafică

Concluzii

- Sistem funcțional de detecție a stărilor de avarie
- Ambele modele utile în contexte diferite
- Aplicație ușor de folosit pentru testare idustrială
- Poate fi extinsă cu predicție în timp real și noi modele

Detecția stărilor de avarie în sisteme industriale folosind tehnici de învățare automată

Student: Andrea Marton
Coordonator: Lector univ. dr. Roland Bolboacă

Mulțumesc de atenție!